Campus-DDoS-SDN: un banco de pruebas basado en Python para la detección de ataques DDoS multivector en redes de campus definidas por software

Anna Olenka Calderón Leiva[a], Ricardo Alexander Llanos Lozano[a], Cristhian Emilio Martínez Casas[a,*] and Alessandro Piero Poves Martínez[a]

[a] *Universidad Nacional del Centro del Perú, , Huancayo, , , Peru*

ARTICLE INFO

Keywords:
Redes definidas por software
Aprendizaje automático
Detección de ataques DDoS
Mininet
Clasificación de tráfico de red

ABSTRACT

Campus-DDoS-SDN es un banco de pruebas en Docker que genera y clasifica cuatro ataques DDoS contra el portal de un campus emulado con Mininet y Ryu. Los alumnos siguen navegando durante el ataque y cada flujo se etiqueta por la IP de origen, no por el archivo. Un bosque aleatorio, entrenado sin direcciones IP y probado en el laboratorio de Ciencias Agrarias, obtiene un F1 macro de 0,979. En la sonda del portal, httpflood: la pagina tarda mas; suben los paquetes al portal; icmpflood: suben los paquetes al portal; slowloris: la pagina deja de abrir; synflood: suben los paquetes al portal. La clase ICMP no se afirma: en la prueba de Agrarias hay 24 flujos.

1. Introducción

Las redes definidas por software (SDN) separan el plano de control del plano de datos, concentrando la inteligencia de la red en un controlador lógicamente centralizado [1]. Esta centralización simplifica la gestión de la red, pero también convierte al controlador en un punto único de falla: un ataque de denegación de servicio distribuido (DDoS) que sature al controlador, a los switches, o a un servicio alojado puede degradar o tumbar todo el segmento de red que gobierna. Detectar estos ataques a tiempo, y separarlos de forma confiable de picos de tráfico legítimo, requiere datasets que sean a la vez representativos de despliegues SDN reales y etiquetados con precisión.

La mayoría de los datasets de detección de intrusiones disponibles públicamente, como NSL-KDD [6], CICIDS2017 [4] y CICDDoS2019 [5], no fueron capturados dentro de un entorno SDN: carecen de señales propias de OpenFlow, como la ocupación de la tabla de flujos o la carga del controlador, y sin embargo suelen reutilizarse fuera de ese contexto para entrenar clasificadores orientados a SDN. Trabajos recientes empezaron a cerrar esta brecha generando datasets de DDoS directamente dentro de bancos de pruebas Mininet/Ryu. Kumar y Gupta publicaron dos datasets de un solo vector construidos así: un dataset de inundación TCP-SYN [2] y, más recientemente, un dataset de inundación ICMP [3] en el que el tráfico normal se captura conectando un host de Mininet a una conexión real a internet y navegando sitios web reales (YouTube, Facebook, noticias, etc.), en vez de depender de generadores de tráfico sintético. Este enfoque de tráfico real aumenta el realismo, pero introduce dos limitaciones prácticas para un benchmark reutilizable: el dataset resultante queda atado a un solo vector de ataque, y no es reproducible entre corridas, ya que el contenido y los tiempos capturados dependen de lo que el internet real ofrezca en el momento de la captura.

Este trabajo, Campus-DDoS-SDN, aborda ambas limitaciones dentro de un único banco de pruebas completamente reproducible. Se lanzan cuatro vectores de ataque (inundación TCP-SYN, inundación HTTP, Slowloris e inundación ICMP) desde hosts ya comprometidos contra una misma víctima compartida, un portal web, dentro de una topología Mininet/Ryu que imita un campus universitario: un esquema de switches core/DMZ más un laboratorio emulado por área académica. A diferencia de los datasets previos de un solo vector o de capturas totalmente separadas, los usuarios legítimos siguen navegando el portal durante todo el ataque, de modo que cada captura mezcla flujos benignos y maliciosos contra el mismo objetivo, tal como ocurriría en un incidente DDoS real. Las etiquetas de verdad fundamental se asignan por IP de origen en vez de por archivo de captura, y cada corrida además registra el efecto del ataque sobre la víctima tal como lo percibe un cliente real (código de respuesta HTTP, latencia, conexiones activas), junto con indicadores a nivel de red (paquetes y bytes por segundo, tamaño de la tabla de flujos OpenFlow). El entorno completo, incluyendo los generadores de tráfico, el portal web propio y el pipeline de etiquetado, se empaqueta como una imagen Docker para eliminar cualquier dependencia de una conexión a internet en vivo o de un sistema operativo de host específico.

Los principales aportes de este trabajo son:

i. Un dataset de DDoS multiclase que cubre cuatro vectores de ataque contra una misma víctima dentro de una topología SDN realista y multisegmento, en vez de un solo tipo de ataque contra una red plana.

ii. Etiquetado por flujo y por IP de origen que mantiene mezclado el tráfico legítimo y malicioso dentro de la misma captura, más cercano a un escenario de ataque

cristhianmartinezcasas@gmail.com (C.E.M. Casas)
ORCID(s):

Table 1
Metadata del código.

Nr.	Descripción	Completar en esta columna
C1	Versión actual del código	v1.0
C2	Enlace permanente al repositorio de esta versión	Completar la URL al publicar
C3	Enlace permanente a la Reproducible Capsule	–
C4	Licencia legal del código	MIT
C5	Sistema de control de versiones usado	git
C6	Lenguajes, herramientas y servicios usados	Python, Bash, Mininet, Open vSwitch, Ryu, CICFlowMeter, Docker
C7	Requisitos de compilación, entornos y dependencias	Docker, Ubuntu 24.04; Python 3.9.25 y 3.12.3
C8	Enlace a documentación para desarrolladores	Ver `README.md` del repositorio
C9	Email de soporte	cristhianmartinezcasas@gmail.com

real que las capturas totalmente separadas usadas en trabajos previos.

iii. Métricas de impacto combinadas a nivel de red (estadísticas de flujo y paquetes vía CICFlowMeter) y a nivel de aplicación (estado HTTP, latencia, saturación de conexiones), capturadas automáticamente en cada corrida.

iv. Un entorno Docker de un solo comando, completamente reproducible, que no depende de una conexión a internet en vivo, resolviendo la limitación de reproducibilidad de los datasets SDN basados en tráfico real.

v. Un clasificador de bosque aleatorio, entrenado sin usar la IP, que en el laboratorio de Ciencias Agrarias (excluido del entrenamiento) alcanza un F1 macro de 0,979, junto con un pipeline preparado para mezclar capturas Wireshark de tráfico benigno de estudiantes (autorizadas) con los ataques solo simulados.

2. Descripción del software

2.1. Arquitectura general

La topología es un campus reducido: módem de borde, switch core, DMZ con el portal y un laboratorio por área académica. Cada laboratorio tiene alumnos y PCs infectadas. Los switches son Open vSwitch en userspace, con Ryu. Durante un ataque los alumnos siguen pidiendo páginas del portal. Desde el navegador del operador, `http://127.0.0.1:8080/` entra como un cliente benigno más (`10.9.0.1`); el ataque no sale del contenedor.

2.2. Funcionalidades del software

`scripts/docker/run_scenario.sh` levanta un escenario (`baseline`, `synflood`, `httpflood`, `slowloris`, `icmpflood`) o los cinco con `all`. `scripts/mininet/capture_campus.py` captura el pcap y la sonda del alumno. `scripts/mininet/pcap_to_csv.py` etiqueta cada flujo por la IP de origen: el alumno queda `BENIGN` y la PC infectada lleva el nombre del ataque. `notebooks/analisis_completo.ipynb` entrena el bosque sin usar la IP y lo prueba en Ciencias Agrarias. `scripts/docker/preparar_hibrido.sh` anonimiza capturas Wireshark autorizadas de estudiantes, las mezcla con los ataques de la simulación y escribe `notebooks/resultados/ir`. El ataque no se ejecuta contra la red real.

Table 2
Detección (bosque aleatorio). F1 macro 0,979.

Clase	Prec.	Recall	F1	Flujos	En el flujo
BENIGN	0,956	0,949	0,953	1775	se distingue bien
httpflood	0,939	0,947	0,943	1458	se distingue bien
icmpflood	1,000	1,000	1,000	24	hay muy pocos flujos para afirmarlo
slowloris	1,000	1,000	1,000	126	se distingue bien
synflood	1,000	1,000	1,000	4708	se distingue bien

3. Panorama de impacto

El dataset de referencia de un solo vector no se puede repetir, porque el tráfico normal sale de internet en vivo [3]. Este banco sí se repite, mezcla alumno y ataque en la misma captura, y ya produce un clasificador. En Ciencias Agrarias, laboratorio que no entró al entrenamiento, el F1 macro es 0,979. La sonda dice qué se siente en la página, no solo qué etiqueta cayó. Con autorización institucional, el mismo pipeline admite tráfico benigno real (Wireshark) sin sacar el ataque del contenedor.

3.1. Proyectos de investigación en curso que usan el software

El banco se usa en el trabajo de tesis de pregrado de los autores en la Universidad Nacional del Centro del Perú. El protocolo de uso de capturas Wireshark de estudiantes está documentado para el encargado de redes en `docs/documento_encargado.docx`.

3.2. Limitaciones y mejoras futuras

El resultado de la tabla sale aún de los alumnos emulados. El dataset híbrido (Wireshark + ataque simulado) queda listo en el código y se actualiza al recibir los pcap autorizados. Cada escenario de esta corrida dura 120 s. ICMP con 120 flujos en todo el conjunto y 24 en la prueba de Agrarias: el F1 perfecto de esa clase no se informa como resultado consolidado. El clasificador no corre dentro de Ryu; trabaja sobre flujos ya capturados. Trabajo futuro:

Table 3
Requisitos del banco de pruebas.

N.°	Descripción	Herramienta	Versión
1	Emulador de red	Mininet	2.3.0
2	Switch virtual	Open vSwitch	3.3.9
3	Controlador SDN	Ryu	4.34
4	Inundación SYN e ICMP	hping3	3.0.0-alpha-2
5	Ataque de baja tasa	slowhttptest	1.9.0
6	Captura de paquetes	tcpdump	4.99.4
7	pcap a CSV	CICFlowMeter	0.5.0
8	Sistema del contenedor	Ubuntu	24.04

repetir la prueba con el fondo Wireshark, un escenario mixto de cuatro vectores a la vez, y más muestras ICMP.

3.3. Publicaciones habilitadas por este software

Aún no hay otra publicación además de este artículo. El software es el que permite esa evaluación.

4. Requisitos de software y hardware

El banco corre en un contenedor Ubuntu 24.04. Las versiones de la tabla se leyeron dentro de la imagen `ataque-sdn`. El modelo se entrenó con Python 3.12.3, scikit-learn 1.9.1 y pandas 3.0.6. Ryu usa Python 3.9.25.

La corrida de este artículo se hizo en un Apple M5 Pro (15 núcleos, 24 GB de RAM, macOS 27.0.1) con Docker Desktop. El contenedor dispuso de 6 CPU y 7,7 GiB de RAM. Open vSwitch trabaja en userspace (`datapath netdev`).

Agradecimientos

Los autores agradecen a la Universidad Nacional del Centro del Perú.

References

[1] Kreutz, D., Ramos, F.M.V., Veríssimo, P.E., Rothenberg, C.E., Azodolmolky, S., Uhlig, S., 2015. Software-defined networking: A comprehensive survey. Proceedings of the IEEE 103, 14–76. doi:10.1109/JPROC.2014.2371999.

[2] Kumar, S., Gupta, S., 2025. SDN TCP-SYN dataset: A dataset for TCP-SYN flood DDoS attack detection in software-defined networks. Data in Brief 59, 111314. doi:10.1016/j.dib.2025.111314.

[3] Kumar, S., Gupta, S., 2026. ICMP-Flood-SDN: A Python based machine learning application for ICMP flood DDoS attack detection in software defined networks. Software Impacts 27, 100819. doi:10.1016/j.simpa.2026.100819.

[4] Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A., 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), pp. 108–116. doi:10.5220/0006639801080116.

[5] Sharafaldin, I., Lashkari, A.H., Hakak, S., Ghorbani, A.A., 2019. Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy, in: Proceedings of the 53rd IEEE International Carnahan Conference on Security Technology (ICCST), pp. 1–8. doi:10.1109/CCST.2019.8888419.

[6] Tavallaee, M., Bagheri, E., Lu, W., Ghorbani, A.A., 2009. A detailed analysis of the KDD CUP 99 data set, in: Proceedings of the 2nd IEEE International Conference on Computational Intelligence for Security and Defense Applications (CISDA).